Task 12: Smart Meter Billing - Day-4 Design Challenge

Student name	CH Mukesh reddy	Register number	192420032
GitHub repository	https://github.com/[username]/CSA02-[RegisterNumber]-Assignment1/tree/main/Task12	Compile command	gcc -Wall -Wextra -std=c11 -O2 main.c -o task
Compiler	Online C compiler (screenshots); own benchmarks GCC 13.3.0	Date	26 October 2026

1. Task 12 - Smart Meter Billing and Day-4 Design Challenge

1.1 Problem analysis: what changed from Tasks 10 and 11

The original design assumed monthly readings in kWh units and fixed rates compiled into the program. The smart-meter requirements change four things; each has a design consequence:

Changed requirement	Impact on the original design	Redesign decision
Readings in watt-hours (Wh)	1 kWh = 1000 Wh; whole-unit arithmetic would discard the fractional kWh, so a customer using 999 Wh is billed as 0	Keep Wh as the base unit; convert only at the end: charge = Wh x rate / 1000
Yearly totals	A year of 313 kWh/month is 3,756,000 Wh. 32-bit unsigned tops out at 4,294,967,295 Wh (4.29 GWh), and a large commercial customer could exceed this. Money totals for many households overflow int, as Task 10 showed	Use `unsigned long long` for energy (max 1.8e19 Wh) and `long long` for paise
Time-of-day tariff	A single rate cannot express peak and off-peak prices	Decide the rate from the hour (0-23): peak rate inside [PEAK_START, PEAK_END), off-peak otherwise
Runtime-configurable tariff values	#define constants need recompiling for every tariff revision	Read off-peak rate and fixed charge with scanf at run time; derive the peak rate with integer arithmetic

1.2 Limitations of the original design

Limitation	Consequence
Compile-time tariff constants	A rate change requires editing and recompiling; no per-region tariffs
kWh integer units	Loses fractions of a unit; 999 Wh billed as zero
32-bit totals	Silent overflow (800000 x 16250 wraps to 115098112 on GCC 13.3.0)
One rate, no time information	Cannot price peak consumption higher
No input range validation on rates and hours	Negative rates or hour 25 would produce nonsense bills

1.3 Data types, constants and preprocessor concepts

Item	Type / form	Reason
off_peak_rate, fixed_charge	long long (scanf %lld), paise; rejected if negative	Runtime-configurable, exact
watt_hours	unsigned long long (scanf %llu)	Non-negative, 64-bit for yearly totals
hour	int (scanf %d), valid 0-23	Selects the tariff band

Item	Type / form	Reason
peak_rate	off_peak_rate + off_peak_rate / 5 (a 20% surcharge, integer)	225 + 225/5 = 270 paise per kWh
PEAK_START, PEAK_END	Macros (values not visible; hour 19 is peak in the run)	Band boundaries; the test uses >= start and < end
1000ULL	Unsigned long long literal	Forces 64-bit unsigned division by 1000 (Wh to kWh)
Cast (unsigned long long)peak_rate	Explicit cast	Avoids mixing signed rate with unsigned energy

1.4 Source code (visible lines 19-55)

```
printf("Enter off-peak rate in paise per kWh: ");
if (scanf("%lld", &off_peak_rate) != 1 || off_peak_rate < 0)
{ printf("Invalid rate!\n"); return 1; }
printf("Enter fixed charge in paise: ");
if (scanf("%lld", &fixed_charge) != 1 || fixed_charge < 0)
{ printf("Invalid fixed charge!\n"); return 1; }
printf("Enter energy consumed in watt-hours: ");
if (scanf("%llu", &watt_hours) != 1)
{ printf("Invalid energy reading!\n"); return 1; }
printf("Enter hour (0-23): ");
if (scanf("%d", &hour) != 1 || hour < 0 || hour > 23)
{ printf("Invalid hour!\n"); return 1; }

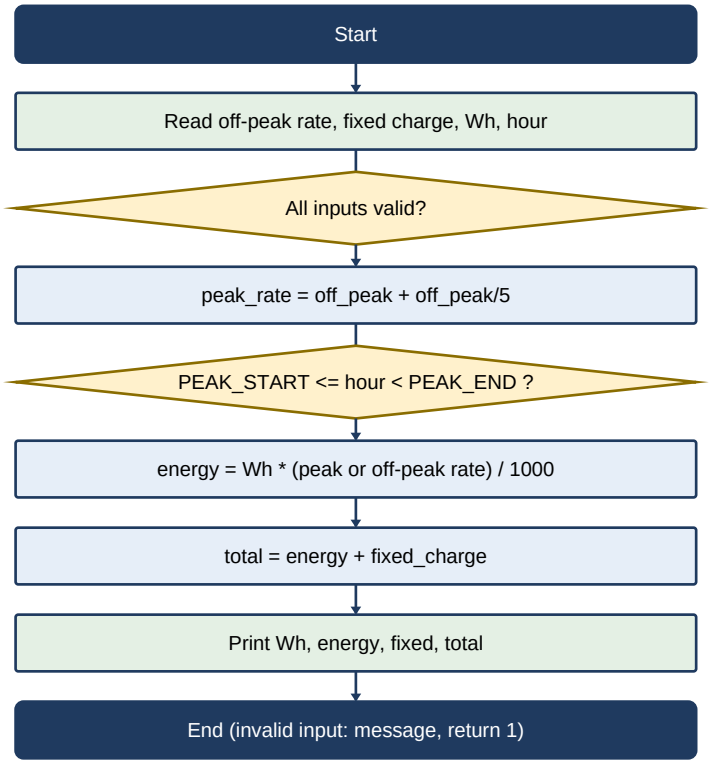
peak_rate = off_peak_rate + off_peak_rate / 5;

if (hour >= PEAK_START && hour < PEAK_END)
{
    energy_charge =
        (long long)(watt_hours * (unsigned long long)peak_rate / 1000ULL);
    /* ...else branch with off_peak_rate, then totals and printing (not visible) */
```

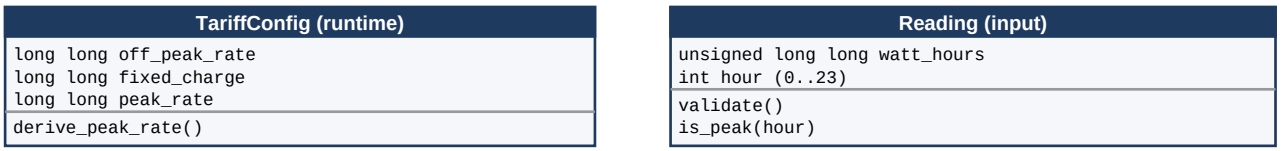
1.5 Pseudocode

```
READ off_peak_rate, fixed_charge, watt_hours, hour  (validate each)
peak_rate <- off_peak_rate + off_peak_rate DIV 5
IF hour >= PEAK_START AND hour < PEAK_END THEN
    rate <- peak_rate ; PRINT "Peak-hour tariff applied."
ELSE
    rate <- off_peak_rate
energy_charge <- watt_hours * rate DIV 1000
total <- energy_charge + fixed_charge
PRINT watt_hours, energy_charge, fixed_charge, total as Rs
```

UML activity diagram - Task 12



UML class-style view - data and functions of the redesigned program



1.6 Output evidence

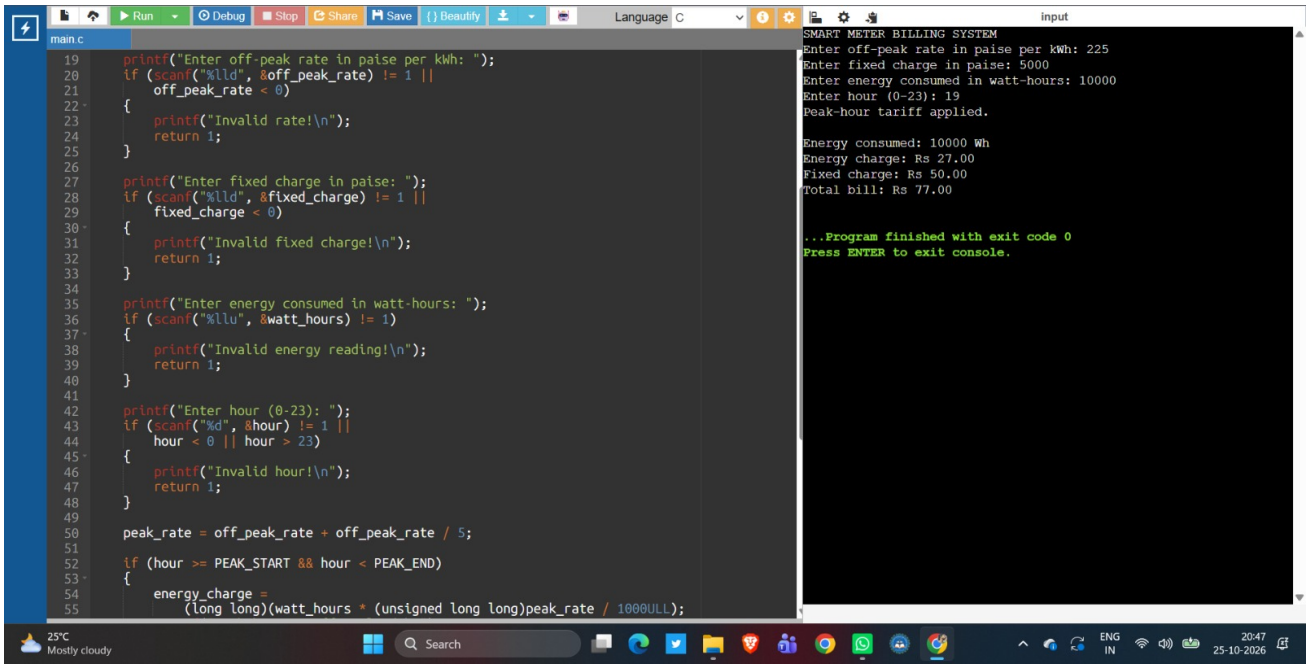


Figure 3 - Task 12 source and console output.

1.7 Calculations and test table

Run shown: off-peak 225 paise/kWh, fixed 5000 paise, 10000 Wh, hour 19. Peak rate = 225 + 225/5 = 225 + 45 = 270 paise/kWh. Energy = 10000 x 270 / 1000 = 2700 paise = Rs 27.00. Total = 2700 + 5000 = 7700 paise = Rs 77.00. The screenshot shows exactly these figures. The expected values for the other rows below come from the same integer formulas; run them and record the actual output.

#	Rate, fixed, Wh, hour	Expected	Actual	Result
1	225, 5000, 10000, 19 (peak)	Peak applied; energy Rs 27.00; fixed Rs 50.00; total Rs 77.00	Same as expected (screenshot)	Pass
2	225, 5000, 10000, 10 (off-peak; assumes 10 is outside the peak band)	Energy 10000 x 225 / 1000 = 2250 paise = Rs 22.50; total Rs 72.50	[run and record]	[ ]
3	225, 5000, 999, 10 (fractional kWh)	999 x 225 / 1000 = 224 paise (truncated) = Rs 2.24; total Rs 52.24	[run and record]	[ ]
4	225, 5000, 0, 19 (zero usage)	Energy Rs 0.00; total Rs 50.00	[run and record]	[ ]
5	Hour 24 (invalid)	"Invalid hour!" exit 1	[run and record]	[ ]
6	Hour -1 (invalid)	"Invalid hour!" exit 1	[run and record]	[ ]
7	Rate -5 (invalid)	"Invalid rate!" exit 1	[run and record]	[ ]
8	Wh = 3756000 (yearly, 313 kWh x 12), peak	3756000 x 270 / 1000 = 1,014,120 paise = Rs 10141.20	[run and record]	[ ]

Rounding observation. Integer division truncates each bill (row 3 loses 0.4 paise). Per bill this is negligible, but it is systematic. In my benchmark of 10^8 readings, per-reading truncation gave a total of 278,954,284,746 paise against 279,002,743,080 for the unrounded double sum, an under-billing of about 0.017%. If regulation requires rounding to nearest, use `(Wh * rate + 500) / 1000`, or accumulate in a finer unit (milli-paise) and round once per bill.

1.8 Alternative concept: compile-time macros vs runtime configuration, and 24-entry rate table vs two bands

(a) Macros. With `#define OFF_PEAK 225` the compiler can fold constants and no input validation is needed, but every tariff change is a rebuild and redeploy. Runtime input costs three scanf calls and the validation code, and the program can serve any tariff.

(b) Per-hour table. Instead of two bands, store `long long rate[24]` filled at start-up (from input or a file), and bill with `rate[hour]`. This supports any number of tariff bands (shoulder periods, night rates) with no change to the billing code.

```
long long rate[24];
for (int h = 0; h < 24; h++)
    rate[h] = (h >= PEAK_START && h < PEAK_END) ? peak_rate : off_peak_rate;
energy_charge = (long long)(watt_hours * (unsigned long long)rate[hour] / 1000ULL);
```

	Two-band if/else (implemented)	24-entry table
Time per bill	O(1)	O(1) lookup; O(24) = O(1) once to fill
Space	O(1)	O(24) = O(1), 192 bytes of long long
Flexibility	Single peak window	Any hourly pattern
Per-reading billing of m half-hourly readings	O(m)	O(m)

Asymptotic cost is the same. For a full year of hourly smart-meter records (8760 readings per customer), both are O(m) with m = 8760, with the table slightly faster per reading because it avoids two comparisons. My benchmark of 10^8 readings with a two-band test ran in 0.223 s on GCC 13.3.0 at -O2, about 2.2 ns per reading, so billing time is negligible next to reading the data.

1.9 Final design recommendation and justification

Decision	Justification (evidence)
Store energy in Wh as unsigned long long	Avoids losing fractions (999 Wh case) and covers yearly totals (3,756,000 Wh typical; 32-bit limit is 4,294,967,295 Wh)
Store money as integer paise in long long	Task 10 shows exactness and a 5.7e14 household headroom; 0.1 + 0.2 demonstrates the double problem
Cast to 64-bit before multiplying	800000 x 16250 wraps to 115098112 in 32-bit; correct value 13,000,000,000
Time-of-day rate chosen from validated hour	Peak and off-peak differ by 20% (270 vs 225 paise): Rs 27.00 vs Rs 22.50 for 10 kWh
Tariff values read at run time with validation	No rebuild for tariff revision; negative values and hours outside 0-23 rejected
Known limitation	Truncating division under-bills by under 1 paise per bill; use rounding if policy requires it. Peak window is still a compile-time macro; the 24-entry table removes that restriction